

Deep Learning HW3

111511141 賴昱凱

1. Construct a standard RNN then show your

(1) network architecture

```
class RNNmodel(nn.Module):
    def __init__(self, input_size = len(vocab), hidden_size = config['hidden_size'], output_size = len(vocab), n_layers = config['n_layers']):
        super(RNNmodel, self).__init__()
        self.hidden_size = hidden_size
        self.n_layers = n_layers

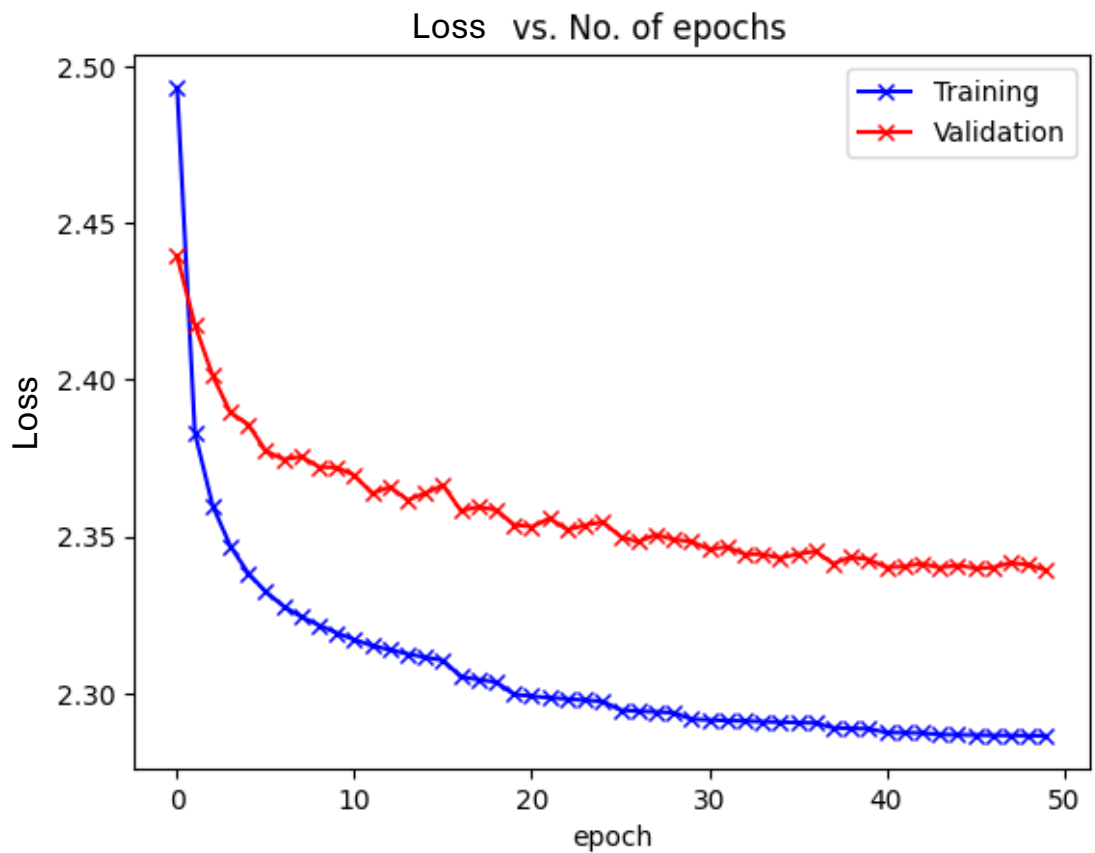
        self.embedding = nn.Embedding(input_size, hidden_size)
        self.rnn = nn.RNN(hidden_size, hidden_size, n_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x, hidden):
        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded, hidden)
        output = output.contiguous().view(-1, self.hidden_size)
        output = self.fc(output)
        return output, hidden

    def init_hidden(self, batch_size):
        return torch.zeros(self.n_layers, batch_size, self.hidden_size).to(config['device'])

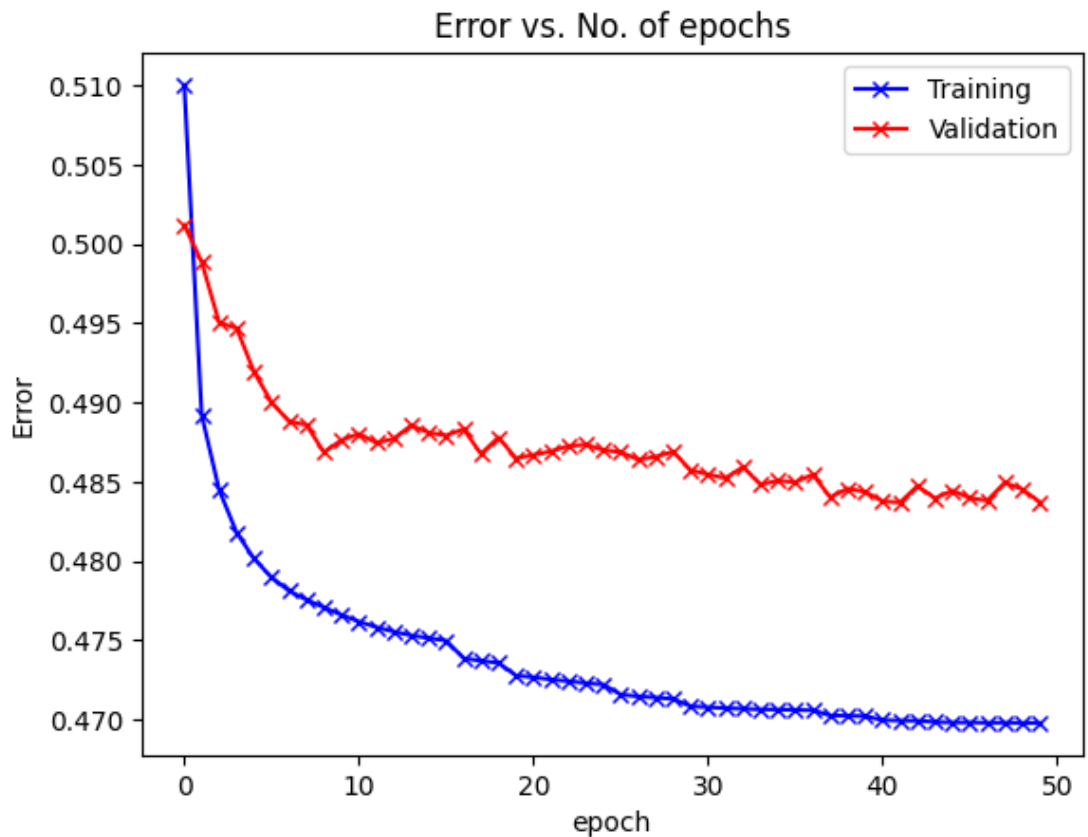
rnn_model = RNNmodel().to(config['device'])
```

(2) learning curve



(3) training error rate

(4) validation error rate



2. Choose 5 breakpoints during your training process to show how well your network learns through more epochs. Feed some part of your training text into RNN and show the text output.

Prime text: speak of n

1) Before training:

speak of nGtNs3k?D:WwQeauClqC3s]elJyKtwNjVTt,aTi sN?[Jk:yqs

2) epoch 1:

speak of neyer.

PUOLIO:

Theart thou that a milder end our b

3) epoch 13

 speak of nothing, nor of
 God comes had that the flights than

4) epoch 35

 speak of nothing lives to Mine or by the tormit?

NE:

Not no

5) epoch 50

 speak of now.

ANTILOUS:

Herlouk.

You she hath the tending o

As observed, before training, the model outputs random letters that do not form any coherent words. However, after just one epoch, the model begins to generate simple words or letter combinations that resemble words. By epoch 13, the model can produce actual words and even correct sentences. With additional epochs, there are no significant differences in the output; the model consistently generates words or short, meaningful sentences. To understand how the model learns to generate coherent text, we set five additional breakpoints within the first epoch.

Prime text: LADY ANNE

1) 0 batches

LADY ANNEGqDFNT:z&UdPFJ:m\$TYf&Um;ov?fK

2) 100 batches

LADY ANNE:

And there igkiedor fale dind se ke to fis, if th

3) 400 batches

LADY ANNE:

Would it were mortal poison, for thy s

4) 700 batches

LADY ANNER:

If no house I wing find the not not all till ch

5) 1000 batches

LADY ANNERICUS:

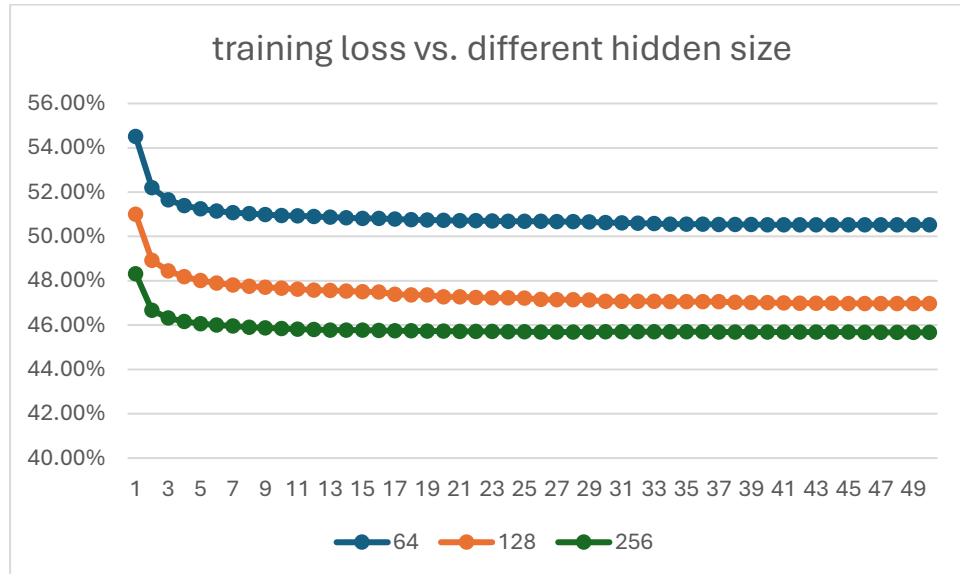
Somer me unto come, be that son that world

As observed, before training, the model produces random letters that fail to form coherent words. However, after just 100 batches, it begins generating some tokens resembling English. By 400 batches, the model understands that "LADY ANNE" refers to a person and appropriately adds a colon ":" after the name. It also starts generating words like "Would" and "it."

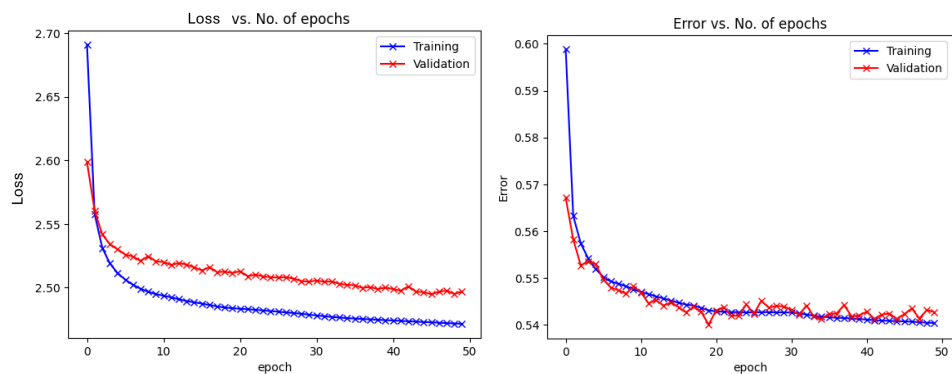
With further training, the model shows no significant improvements in output quality, though it consistently generates words or short, meaningful sentences. Nonetheless, many grammatical and contextual errors persist in its sentences.

3. Compare the results of choosing different sizes of hidden states and sequence length by plotting the training loss vs. different parameters.

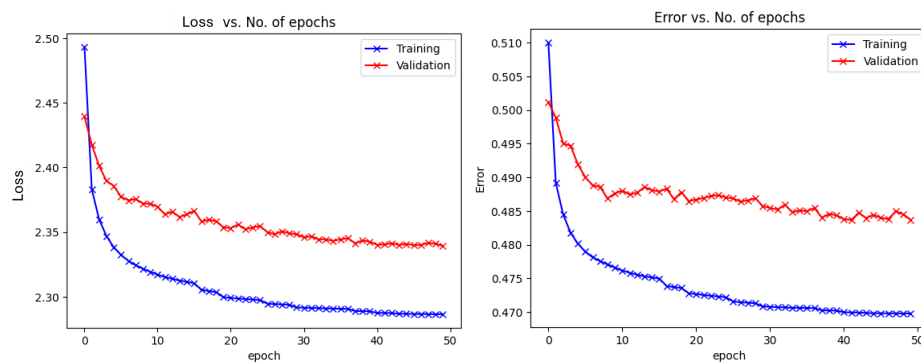
1) Different hidden size



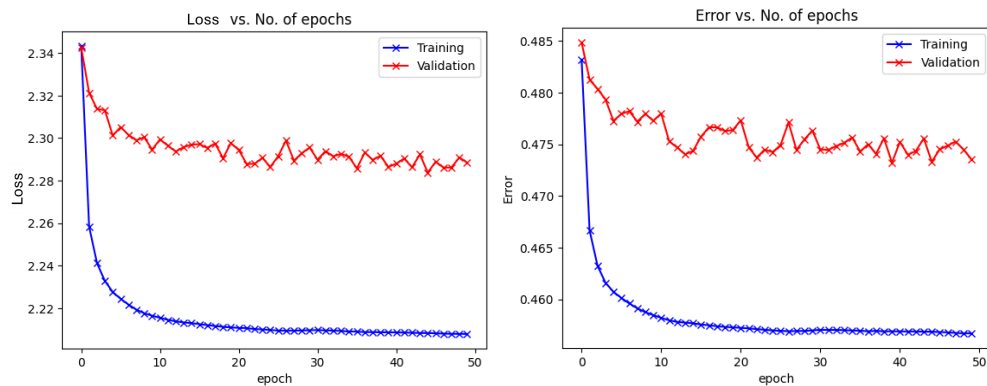
a. hidden size = 64



b. hidden size = 128

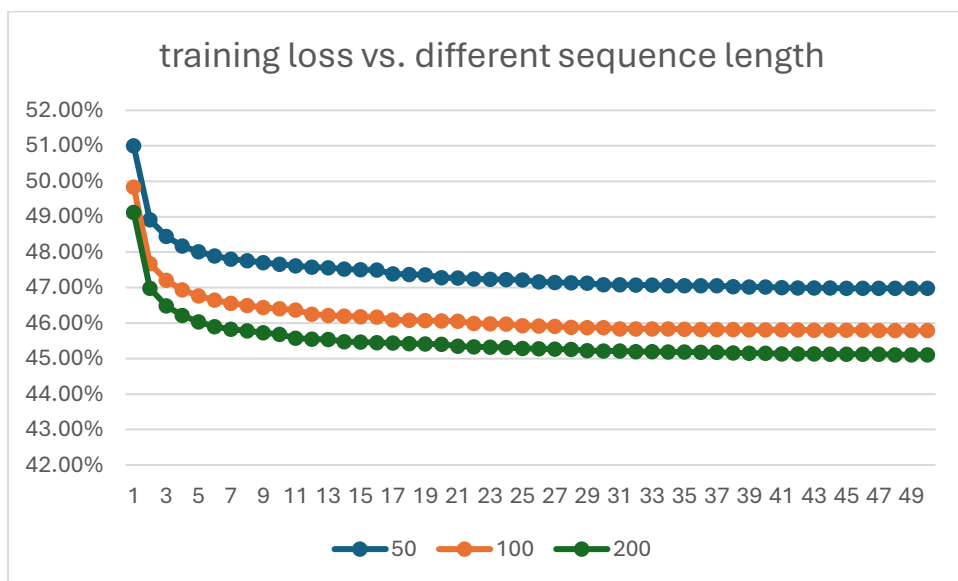


c. hidden size = 256

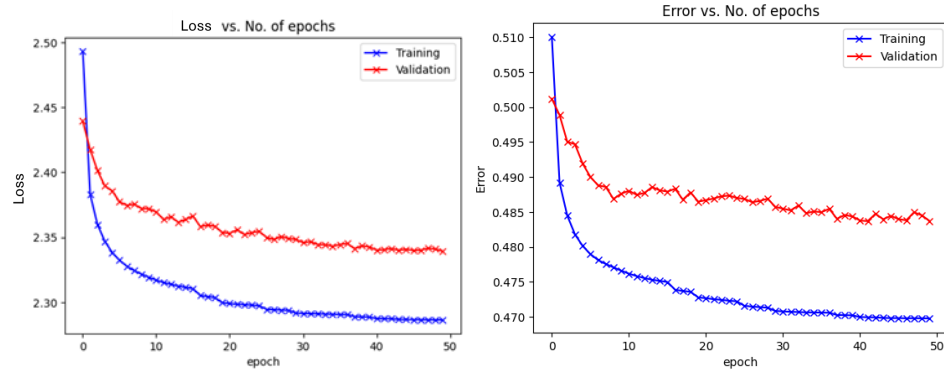


According to the plot above, it is evident that as the hidden size increases, the training error decreases. However, the generalization ability behaves differently: smaller hidden sizes tend to improve generalization, while larger hidden sizes increase the probability of overfitting. This indicates a trade-off between reducing training error and maintaining good generalization performance.

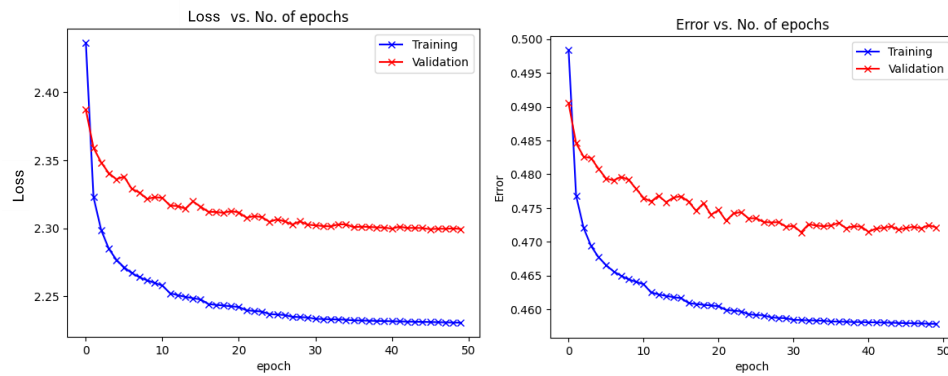
2) Different sequence length



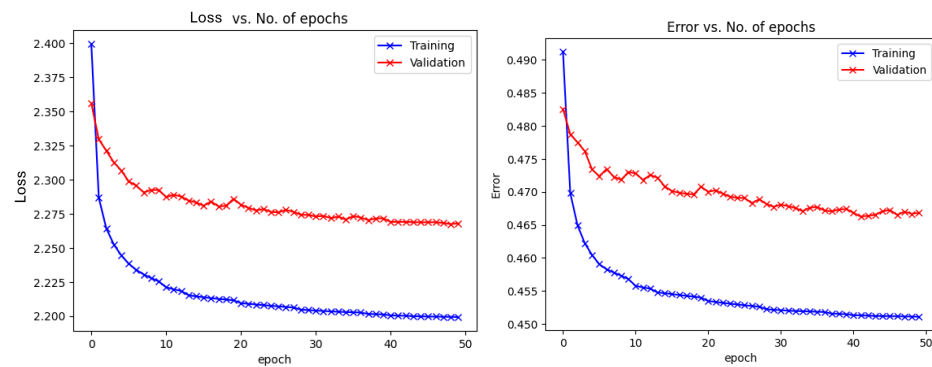
a. 50



b. 100



c. 200



According to the plot above, as the sequence length increases, the training error decreases slightly. This is likely because the model has more data to learn character combinations. However, the improvement is not very significant.

Regarding the validation error, the model also faces generalization issues. As the sequence length decreases, its generalization ability tends to decline, indicating that shorter sequences provide less context for the model to learn meaningful patterns.

4. Construct a LSTM then show your

(1) network architecture

```
class LSTMModel(nn.Module):
    def __init__(self, input_size = len(vocab), hidden_size = config['hidden_size'], output_size = len(vocab), n_layers = config['n_layers']):
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.n_layers = n_layers

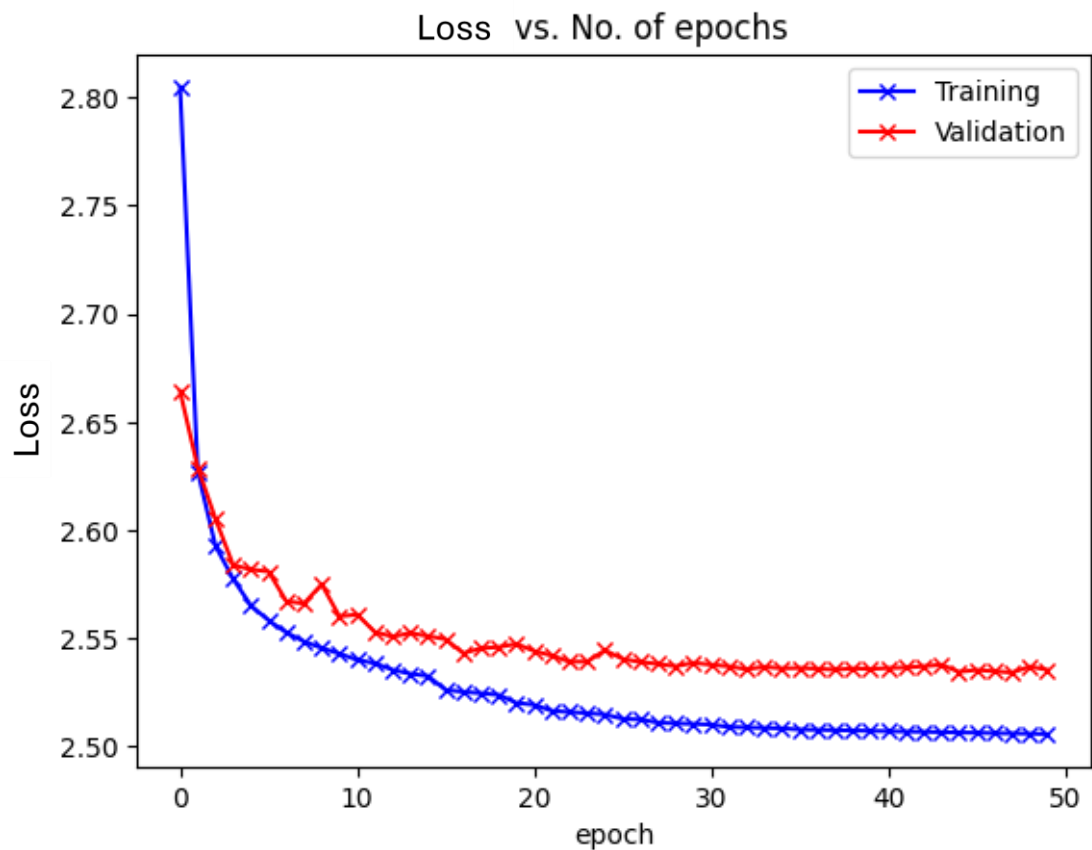
        self.embedding = nn.Embedding(input_size, hidden_size)
        self.lstm = nn.LSTM(hidden_size, hidden_size, n_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x, hidden):
        embedded = self.embedding(x)
        output, hidden = self.lstm(embedded, hidden)
        output = output.contiguous().view(-1, self.hidden_size)
        output = self.fc(output)
        return output, hidden

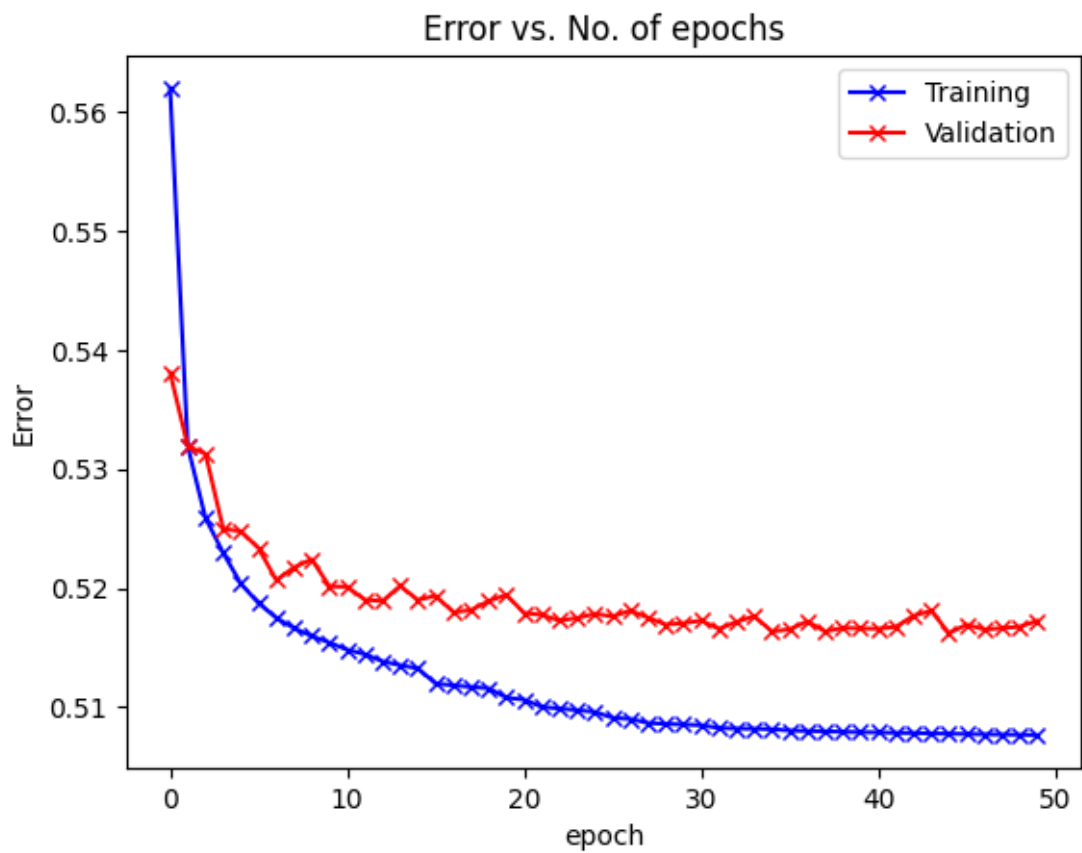
    def init_hidden(self, batch_size):
        return (torch.zeros(self.n_layers, batch_size, self.hidden_size).to(config['device']),
                torch.zeros(self.n_layers, batch_size, self.hidden_size).to(config['device']))

lstm_model = LSTMModel().to(config["device"])
```

(2) learning curve



- (3) training error rate
- (4) validation error rate



5. Choose 5 breakpoints during your training process to show how well your network learns through more epochs. Feed some part of your training text into LSTM and show the text output.

Prime text: speak of n

- 1) Before training

speak of nNnP-moco&k!Bthn3::[K: L
XgoPykkpRvesgxHRsl-yW3JBDP

- 2) Epoch 1

speak of night of more so heagtters,
You or rown his apted t

3) Epoch 13

 speak of nebless have this is whose,
 My roon to the juill. L

4) Epoch 35

 speak of noldiders; I am his montage thas spaise,
 Wether to

5) Epoch 50

 speak of nor my boot-groness,
 Who sheacuse the clook of leas

Just like RNN model, before training, the model outputs random letters that do not form any coherent words. However, after just one epoch, the model begins to generate simple words or letter combinations that resemble words. By epoch 13, the model can produce actual words and even correct combination of words. With additional epochs, there are no significant differences in the output; the model consistently generates words or short, meaningful sentences. To understand how the model learns to generate coherent text, we set five additional breakpoints within the first epoch.

The most significant difference I observed between RNN and LSTM lies in their performance. In my experience, RNN outperforms LSTM, likely because the training time was insufficient for LSTM to converge. Since RNN converges more quickly, it may have reached a better state within the same training duration.

Prime text: Claudio l

1) 0 batches

 Claudio l\$Awq;,M:VQwlrEQlxoTgTj3fCzCPkbbjr W

Le3.vIhilCAgl

2) 100 batches

Claudio l choow teorn hetoo!

I

Rbtsp y.

Oo tfhoa afeeesai

3) 400 batches

Claudio lem

on an feost nos -our ald her;

Wish lof by than

4) 700 batches

Claudio lept as modte havire, the cain

Fout to the with por

5) 1000 batches

Claudio loth on;

Beounce other thy see sake and the that dr

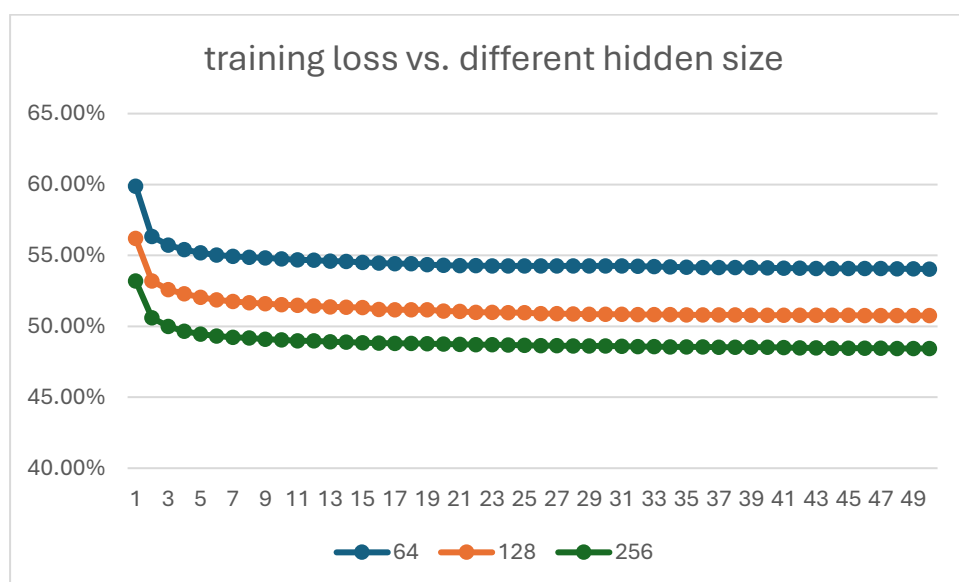
It is also similar to RNN model, before training, the model produces random letters that fail to form coherent words. However, after just 100 batches, it begins generating some tokens resembling English. By 400 batches, the model starts generating simple words like “Wish”, “on”, “than” ...

With further training, the model shows no significant improvements in output quality, though it consistently generates words or short, meaningful sentences. Nonetheless, many grammatical and contextual errors persist in its sentences.

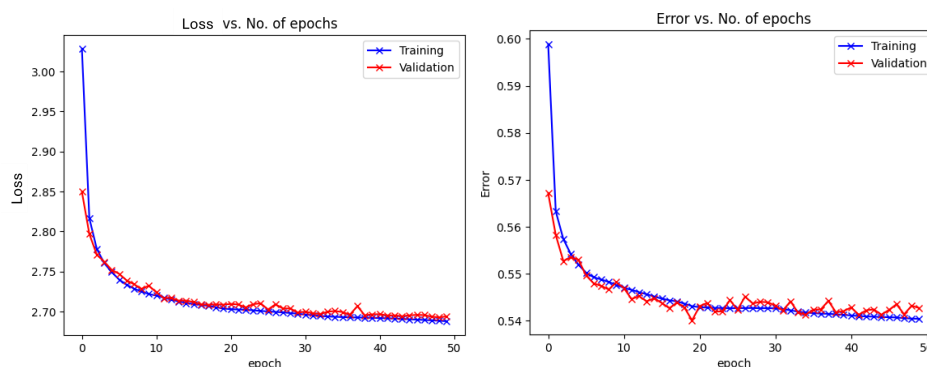
The most notable difference between RNN and LSTM lies in the length of the generated words. After 100 batches, LSTM tends to produce longer, more coherent “words,” whereas RNN typically generates shorter “tokens.” This suggests that LSTM may capture longer-term dependencies more effectively, even in the early stages of training.

6. Compare the results of choosing different sizes of hidden states and sequence length by plotting the training loss vs. different parameters.

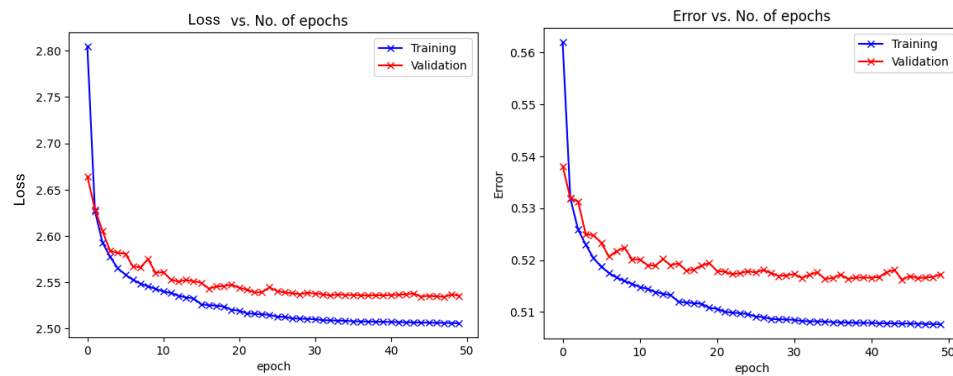
1) Different hidden size



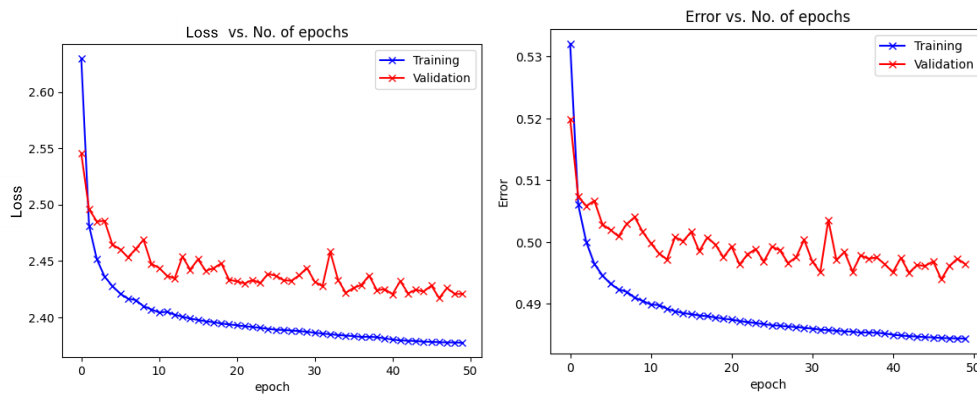
a. hidden size = 64



b. hidden size = 128

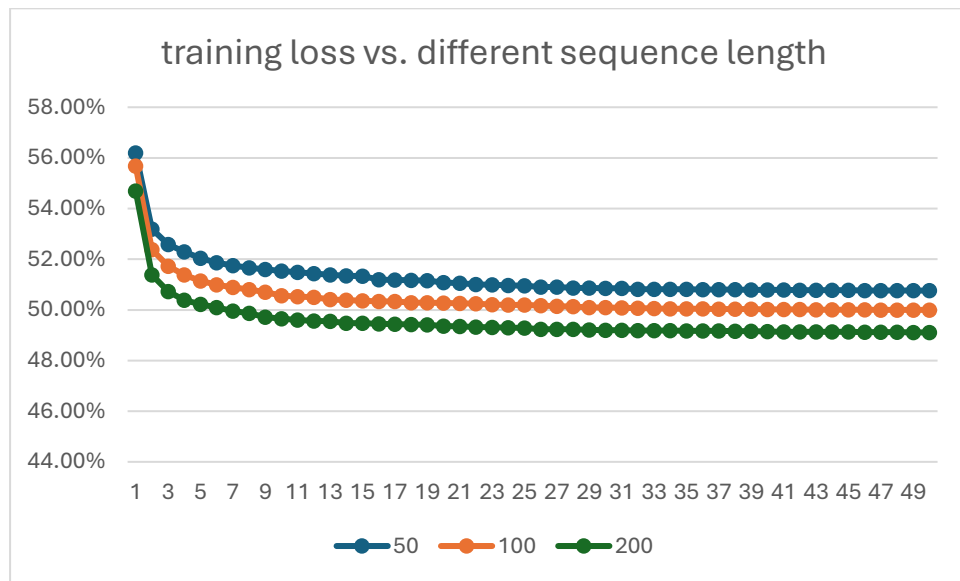


c. hidden size = 256

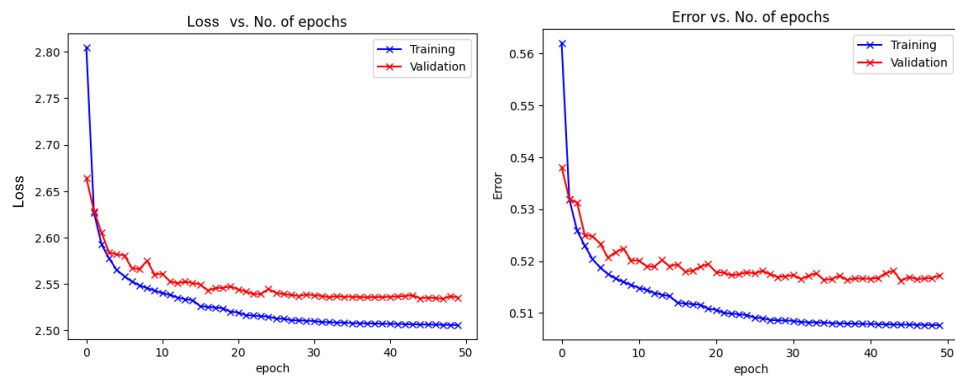


According to the plot above, it is evident that as the hidden size increases, the training error decreases. However, the generalization ability behaves differently: smaller hidden sizes tend to improve generalization, while larger hidden sizes increase the probability of overfitting. This indicates a trade-off between reducing training error and maintaining good generalization performance.

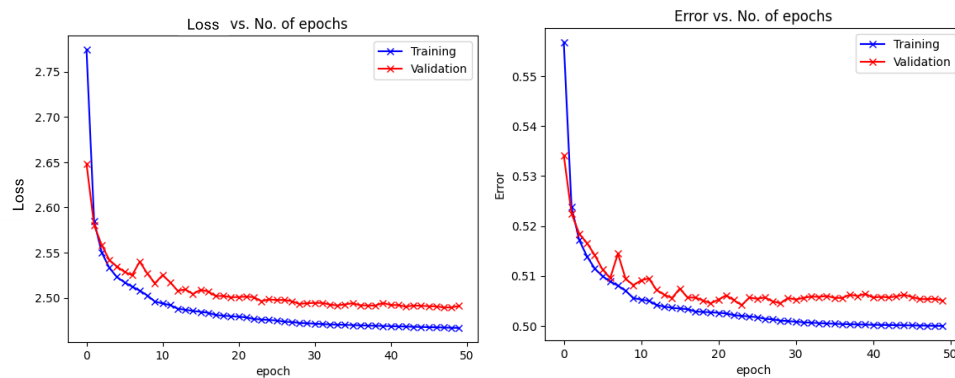
2) Different sequence length



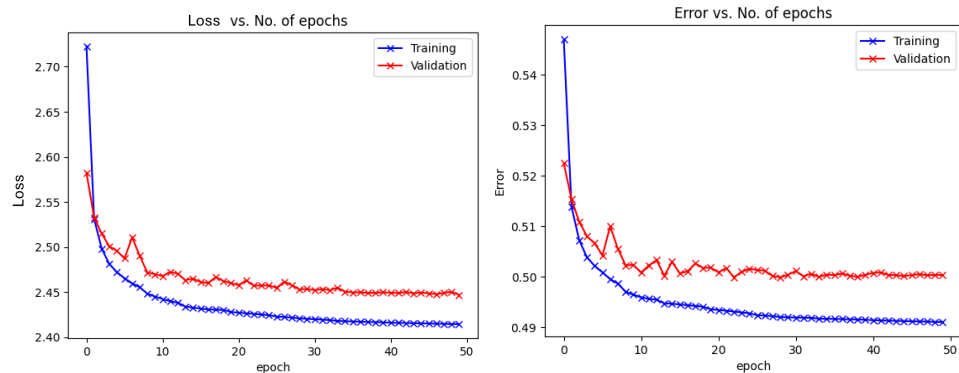
a. 50



b. 100



c. 200



According to the plot above, as the sequence length increases, the training error decreases slightly, though the improvement is not very significant.

In terms of validation error, the model performs significantly better than traditional RNN models. Unlike RNNs, where validation error often diverges from training error, this model maintains validation error consistently close to training error, indicating a high degree of generalization and better performance on unseen data.

However, RNN still outperforms LSTM in overall performance. Nonetheless, as the sequence length increases, the performance gap between RNN and LSTM is slightly smaller.

7. Use RNN to generate some words by priming the model with a word related to your dataset.

Prime text: JULIET

JULIET:

The change is a heaven any awafe,
Which hath the tone to me, their together to the
Toole of nemvers the sweet have best duty heaven.

PRONTES:

If your holy dayghter good like and, that all,
In the fear, and how This shall or than I have
never that with needs and all thus with a man.

AUFOLINE:

Now thou may, the dather eatest with my day
For seels for provided to your vesserful,
And what he remed me, to gavalitis
And meet you are upon thee washed here,
This assians of night and before the wife That bear't the hustred